gridctl





Put your agent setup in version control.

MCP servers, skills, agent definitions, and rules, treated as disposable infrastructure.

William Collins · October 2026 · wcollins.io

01

Why gridctl

MCP servers are everywhere: different transports, different hosting models, different JSON files.

Agenda

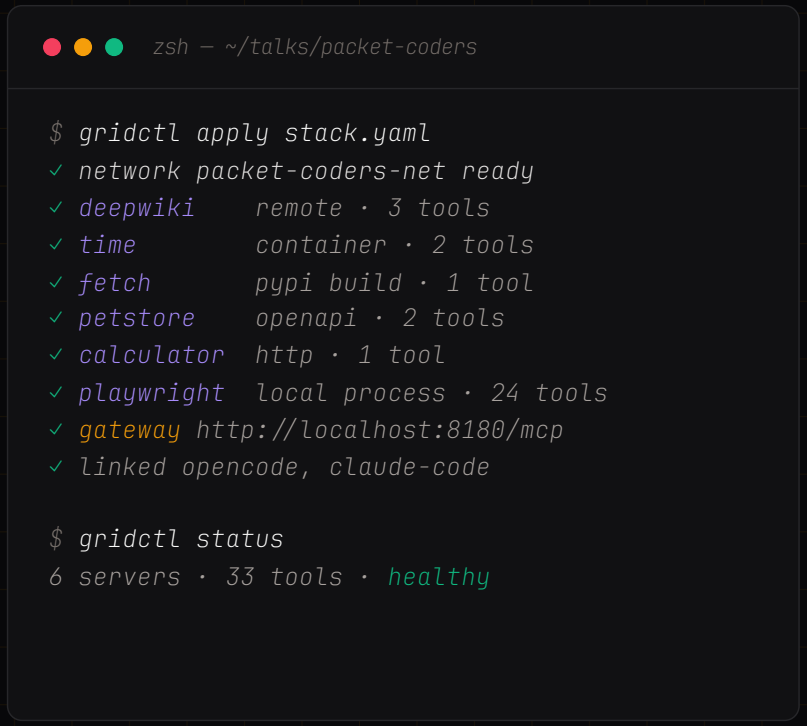
01 **The problem**
One JSON file per client, secrets in plaintext, nothing to review

02 **One file**
Validate, plan, apply, and destroy a stack.yaml, live

03 **Six kinds of server**
Remote URL, container image, built from source, OpenAPI, local process, SSH

04 **Control the surface**
Whitelists, groups, client scopes, rate limits, and code mode

05 **Observe**
Logs, traces, metrics, pins, connections, and the library



gridctl

Stack as code

One declarative file for the whole agent setup. Validate before you commit, plan before you apply, and let the gateway flag drift when what is running stops matching the spec.

One file
Servers, secrets, groups, limits, and client links in a single stack.yaml

Plan, then apply
gridctl plan diffs the spec against running state before anything changes

Disposable
destroy --unlink removes the stack and its client entries; apply brings it back

stack.yaml

```yaml
version: "1"
name: packet-coders

link: [opencode, claude-code]

mcp-servers:
  - name: deepwiki
    url: https://mcp.deepwiki.com/mcp
  - name: time
    image: mcp/time@sha256:9c46a918…
    transport: stdio
  - name: fetch
    source: { type: pypi, package: mcp-server-fetch, ref:
2026.8.18 }
  - name: github
    url: https://api.githubcopilot.com/mcp/readonly
    auth: { type: bearer, token: ${var:GITHUB_PAT} }
```

Six kinds of server, one file

Remote URL

url: over Streamable HTTP or SSE. Nothing to run. Bearer, header, or brokered OAuth for auth.

Container image

image: pinned by digest, stdio or HTTP. Docker or Podman, auto-detected.

Built from source

source: an exact PyPI release, a commit-pinned git checkout, or a local Dockerfile.

OpenAPI

openapi: a REST spec becomes tools. Filter operations before they are generated.

Local process

command: on the host over stdio. npx, uvx, or a venv path, relative to the file.

SSH

ssh: plus command. A process on another machine, with jump host and known_hosts support.

What the gateway does

Aggregate

Every MCP server, whatever its transport or host, behind one local endpoint.

Skills, agents, rules

SKILL.md, agent definitions, and rule fragments, projected to the clients you select.

Control the surface

Per-server whitelists, tool groups, client scopes, and rate limits.

Code mode

Hide large tool lists behind search and execute meta-tools to save context.

Observe

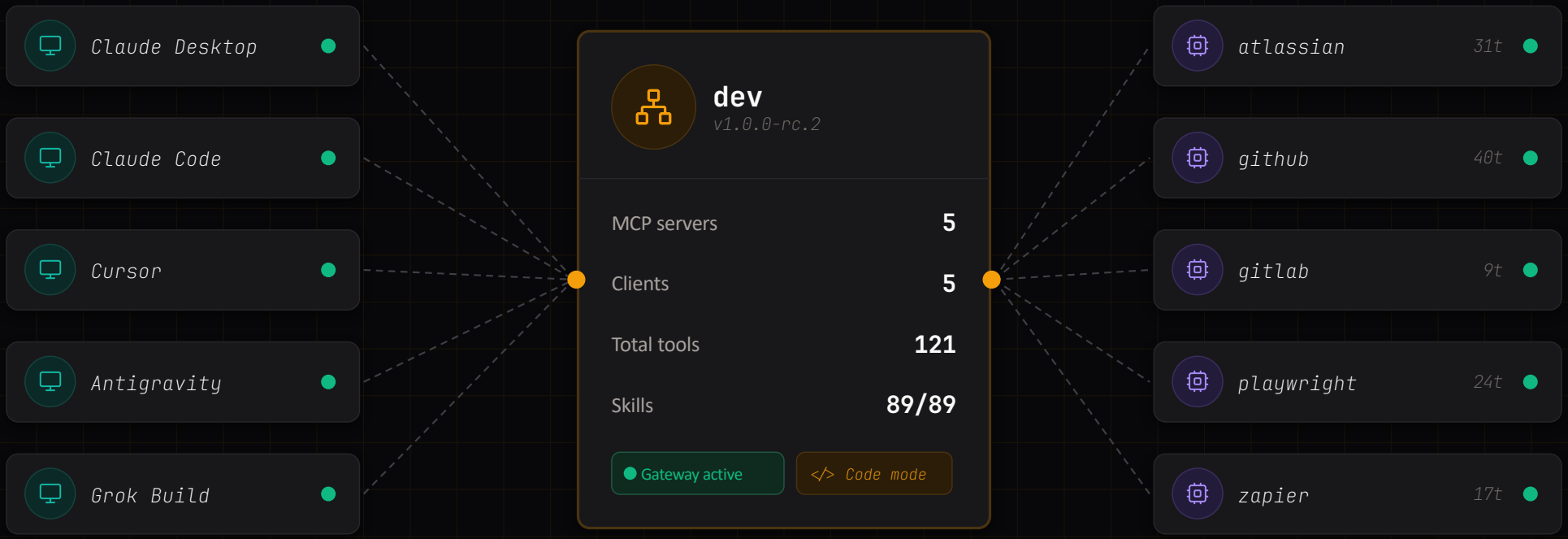
Token metrics, traces, and gridctl optimize with paste-ready remediations.

Link clients

Detects 16 LLM clients and writes the gateway entry, with ownership recorded.

gridctl

Architecture



The demo loop

zsh

```
# Install the latest release to ~/.local/bin/gridctl
$ curl -fsSL https://raw.githubusercontent.com/gridctl/gridctl/main/install.sh | sh

# Validate the file, preview the diff, converge. Repeat after every uncomment.
$ gridctl validate stack.yaml
$ gridctl plan stack.yaml
$ gridctl apply stack.yaml          # first time
$ gridctl reload packet-coders      # every edit after that

# Prove it from the terminal, then from a client
$ gridctl tools search time
$ gridctl call time__get_current_time '{"timezone":"Asia/Tokyo"}'
$ gridctl open

# Tear it down, client entries included. Rebuild it tomorrow.
$ gridctl destroy stack.yaml --unlink
```

Before and after

Before

One JSON file per client, edited by hand

- Servers duplicated across Claude, Cursor, and VS Code configs
- API keys and tokens sitting in plaintext on disk
- Every client sees every tool, so context fills with noise
- No record of what is actually running versus what was declared

After

One stack.yaml, one endpoint, every client linked

- gridctl link writes the gateway entry into 16 supported clients
- Secrets resolve from an encrypted variable store at apply time
- Groups, client scopes, and limits narrow the surface per consumer
- Validate, plan, pins, and drift detection keep spec and runtime aligned

The apply loop



`gridctl validate`

Lint and schema-check the file. Exit 0 clean, 1 error, 2 warning, so CI can gate it

`gridctl plan`

Diff the file against what is running and show the delta. Nothing changes yet

`gridctl apply`

Converge: start containers and processes, wire the gateway, link clients. Later edits hot reload

`gridctl optimize`

Read gateway usage and name the servers and tools nobody has called

`gridctl destroy`

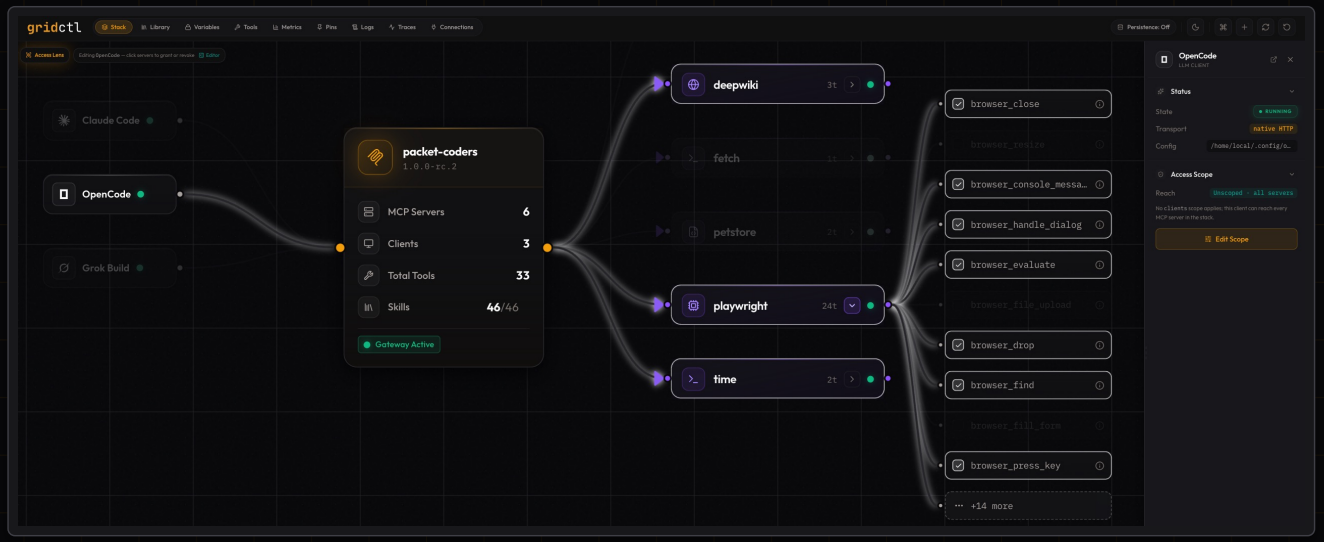
Tear it down. --unlink also pulls the entries back out of client configs

> MCP servers, skills, agent definitions, and rules belong in files.

Running processes, secret values, and client installs do not. Inspired by Containerlab.

README · github.com/gridctl/gridctl

The stack canvas



1. **Clients on the left**
   Each linked LLM client appears as a node with a live health dot.

2. **Gateway in the center**
   Counts for tools, resources, sessions, clients, and skills plus the code-mode badge.

3. **Servers and tools on the right**
   Expand a server to see every tool it contributes to the endpoint.

Where dashboards shine



Logs

What happened. Structured gateway and server output, redacted, grep-able, tailed with gridctl logs.



Traces

How it happened. One tool call fanned through the gateway, with timing on every hop.



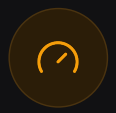
Metrics

How much. Tokens in and out per server, tool, and client. Context is the scarce resource.



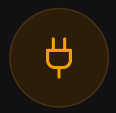
Pins

Did the ground move. Tool schemas are pinned on first use; a changed description trips the wire.



Connections

What is wired on this machine. Every detected client and exactly what gridctl owns in it.



Library

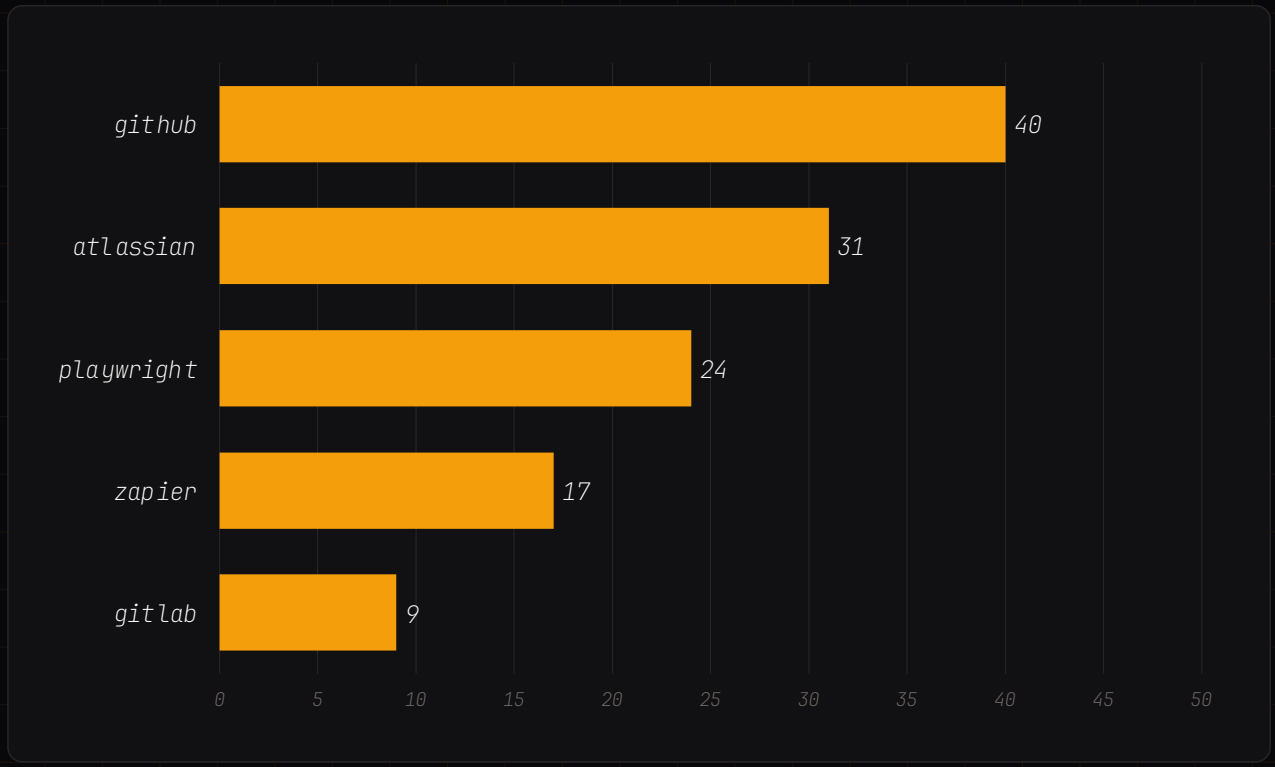
Skills, agents, rules, and packs. The prose half of the setup, projected to the clients that read files.

Feature stability

Feature	Status	Compatibility
MCP gateway (stdio, SSE, HTTP)	Stable	Backward compatible in 0.x
Container orchestration (Docker, Podman)	Stable	Backward compatible in 0.x
Generated Python source containers	Stable	Opt-in through source.runtime: python or source.type: pypi
Code mode	Stable	Failure logs use local categories without source excerpts
Live tool invocation (gridctl call)	Implemented	Included in v1.0.0-rc.2; talks to a selected running gateway
Outbound MCP-to-A2A adapter	Experimental	Off by default; 24-hour handle expiry
MCP execution controls	Implemented	Included in v1.0.0-rc.2; rootless Podman acceptance pending

Source: docs/project-status.md at v1.0.0-rc.2

Tools per server



121

tools behind one endpoint

This is my daily stack. A large stack floods the client with tools it will never call.

Three axes compose: the per-server tools whitelist, groups with their own endpoint, and client scopes.

gridctl

Try it today.

Apache 2.0. Stars, issues, and pull requests welcome.

```
$ curl -fsSL https://raw.githubusercontent.com/
    gridctl/gridctl/main/install.sh | sh
```

github.com/gridctl/gridctl · wcollins.io